



(19) **United States**

(12) **Patent Application Publication**
Salonidis et al.

(10) **Pub. No.: US 2025/0278474 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **BYZANTINE-RESILIENT BILEVEL
FEDERATED LEARNING**

(52) **U.S. Cl.**

CPC **G06F 21/55** (2013.01)

(71) Applicants: **International Business Machines Corporation**, Armonk, NY (US);
Rensselaer Polytechnic Institute, Troy, NY (US)

(57)

ABSTRACT

An intermediate global lower-level machine learning model is generated by executing federated learning model training using data from an identified first set of lower-level non-Byzantine client computers which are characterized as being non-Byzantine for the lower-level system. A global lower-level machine learning model is generated by executing the federated learning model training using data from an identified second lower-level set of non-Byzantine client computers which are characterized as being non-Byzantine for the lower-level system. An intermediate global upper-level machine learning model is generated by executing the federated learning model training using data from an identified first upper-level set of non-Byzantine client computers which are characterized as being non-Byzantine for an upper-level system. A global upper-level machine learning model is generated by executing the federated learning model training using data from a second upper-level set of client computers which are characterized as being non-Byzantine for the upper-level system.

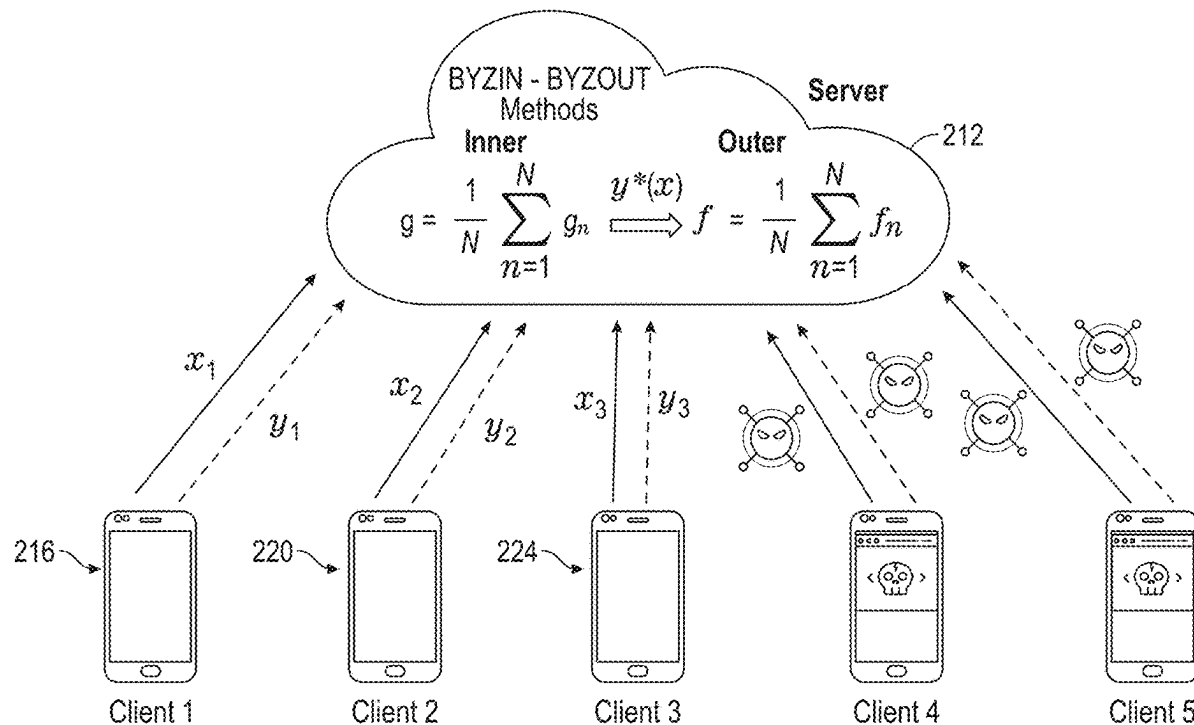
(72) Inventors: **Theodoros Salonidis**, Wayne, PA (US);
Yi Zhou, San Jose, CA (US); **Momin Abbas**, New York, NY (US); **Parikshit Ram**, Atlanta, GA (US); **Nathalie Baracaldo Angel**, San Jose, CA (US);
Horst Cornelius Samulowitz, Armonk, NY (US); **Tianyi Chen**, Rensselaer, NY (US)

(21) Appl. No.: **18/595,386**

(22) Filed: **Mar. 4, 2024**

Publication Classification

(51) **Int. Cl.**
G06F 21/55 (2013.01)



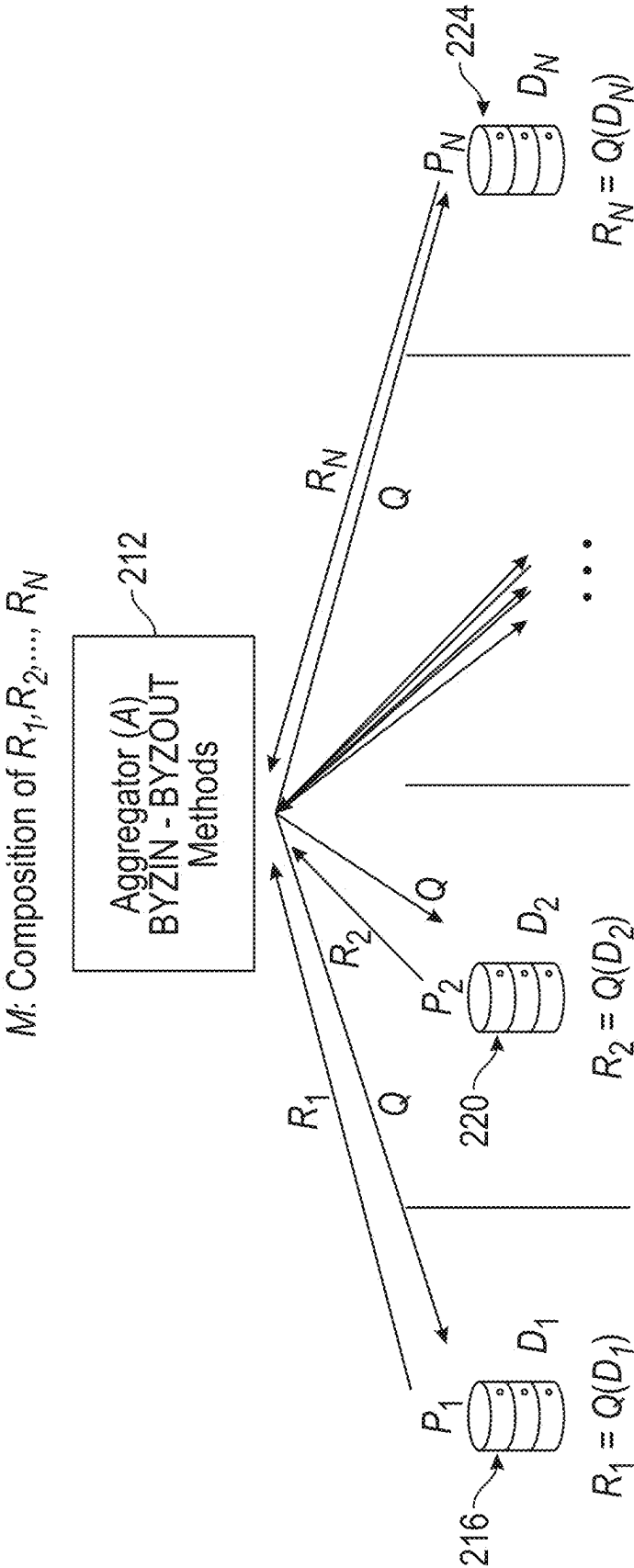


FIG. 1

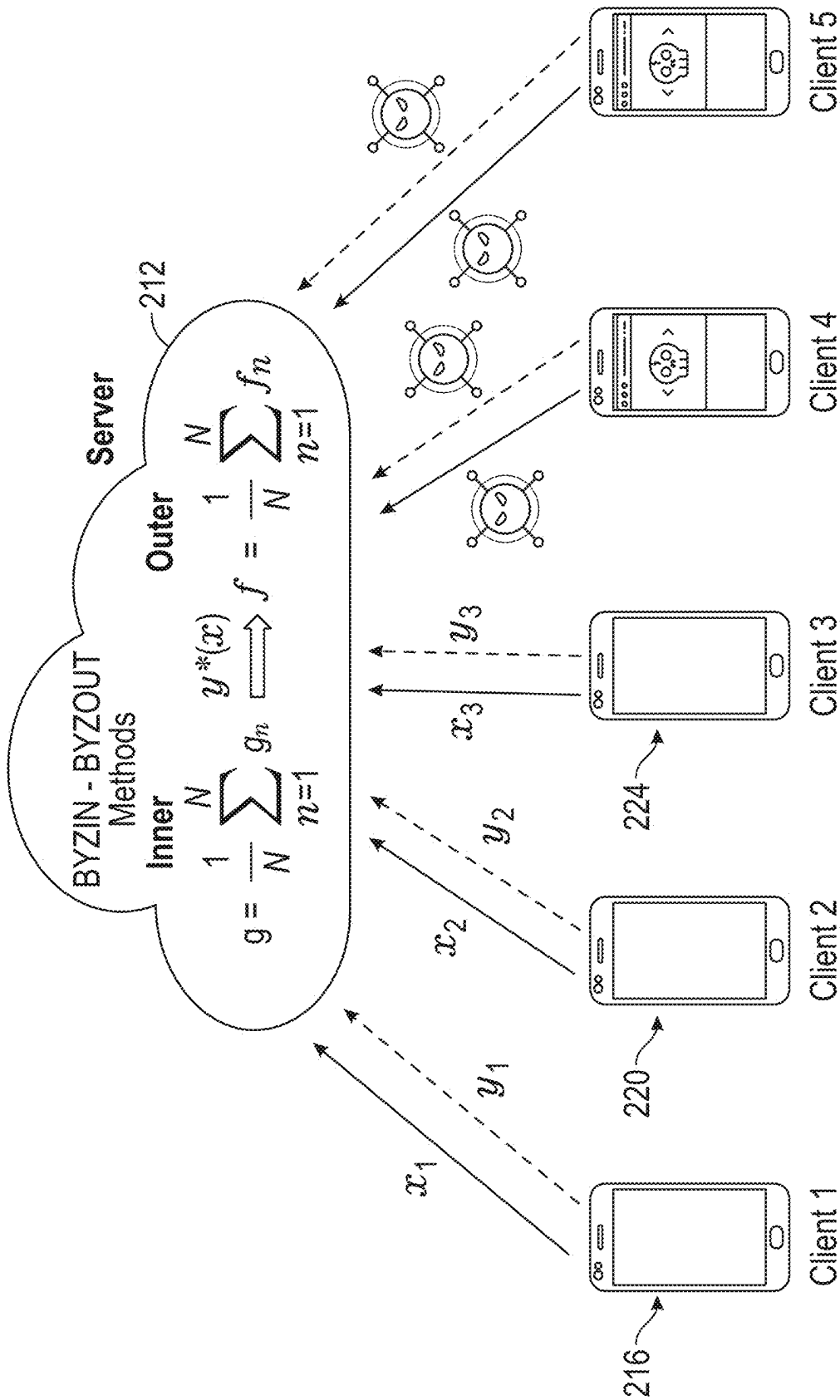


FIG. 2

Algorithm 1 BILANTINE

Input: $K, T \in \mathbb{N}^+$; x^0, y^0 , stepsizes $\{(\alpha^k, \beta^k)\}_{k=0}^{K-1}$,
 assumed ratio of upper/lower Byzantine clients: η_u, η_l
for $k = 0, \dots, K - 1$ **do**
 $y^{k,0} = y^k$
 for $t = 0, \dots, T - 1$ **do**
 $y^{k,t+1} = \text{BYZIN}(x^k, y^{k,t}, \beta^k, \eta_l)$
 end for
 $y^{k+1} = y^{k,T}$
 $x^{k+1} = \text{BYZOUT}(x^k, y^{k+1}, \alpha^k, \eta_u)$
end for

FIG. 3

Algorithm 2 : $x^+ = \text{BYZOUT}(x, y^+, \alpha, \eta_u)$.

Initialize: $\alpha_i \in (0, \alpha]$
Select $R \in \mathbb{N}$ **and set** $P_R = \text{InvHG}(x, y^+, R)$
for all clients $i \in \mathcal{N}$ **in parallel do**
 Compute $h_i = h_i(x, y^+)$ **via (7)**
end for
 $\bar{h} = \text{RES}(\{h_1, \dots, h_N\}, \eta_u)$
for all clients $i \in \mathcal{N}$ **in parallel do**
 for $v = 0, \dots, T_i - 1$ **do** $\triangleright$ **Set** $x_{i,0} = x$
 $h_{i,v} = \nabla_x f_i(x_{i,v}^k; \xi_{i,v}) - \nabla_x f_i(x; \xi_{i,v}) + \bar{h}$
 $x_{i,v+1} = x_{i,v} - \alpha_i h_{i,v}$
 end for
end for
 $x^+ = \text{RES}(\{(x_1, T_1), \dots, (x_N, T_N)\}, \eta_u)$

FIG. 4

Algorithm 3 : $y^+ = \text{BYZIN}(x, y, \beta, \eta_l)$.

Initialize: $\beta_i \in (0, \beta]$
 for all clients $i \in \mathcal{N}$ in parallel do
 $q_i = \nabla_y g_i(x, y; \zeta_i)$
 end for
 $\bar{q} = \text{RES}(\{q_1, \dots, q_N\}, \eta_l)$
 for all clients $i \in \mathcal{N}$ in parallel do
 for $v = 0, \dots, \tau_i - 1$ do $\triangleright$ Set $y_{i,0} = y$
 $q_{i,v} = \nabla_y g_i(x, y_{i,v}; \zeta_{i,v}) - \nabla_y g_i(x, y; \zeta_i) + \bar{q}$
 $y_{i,v+1} = y_{i,v} - \alpha_i q_{i,v}$
 end for
 end for
 $y^+ = \text{RES}(\{(y_{1,\tau_1}), \dots, (y_{N,\tau_N})\}, \eta_l)$

FIG. 5

Algorithm 4 : $P_{R'} = \text{InvHG}(x, y^+, R)$.

Select $R' \in \{0, \dots, R\}$ and $\mathcal{N}_0 \in \mathcal{N}$ UAR
 for all $i \in \mathcal{N}_0$ in parallel do
 $p_{i,0} = \nabla_y f_i(x, y^+; \xi_{i,0})$
 end for
 $p_0 = \frac{R}{l_{g,1}} |\mathcal{N}_0|^{-1} \sum_{i \in \mathcal{N}_0} p_{i,0}$
 if $R' = 0$ then
 return $p_{R'}$
 end if
 Select $\mathcal{N}_1, \dots, \mathcal{N}_{R'} \in \mathcal{N}$ UAR.
 for all $r = 1, \dots, R'$ do
 for $i \in \mathcal{N}_r$ in parallel do
 $p_{i,r} = (1 - \frac{1}{l_{g,1}} \nabla_y^2 g_i(x, y^+; \zeta_{i,r})) p_{i,r-1}$
 end for
 $p_r = |\mathcal{N}_r|^{-1} \sum_{i \in \mathcal{N}_r} p_{i,r}$
 end for

FIG. 6

Algorithm 5 $g^+ = \text{RES}(G, \eta_x)$.

Input: $G = \{G_1, \dots, G_N\}$, η_x

Initialize: $j = 1$

while $j < \eta_x N$ **do**

$G_j = |G|$

Compute $g^{av} = \frac{1}{G_j} \sum_{i=1}^{G_j} G_i$

Compute $G^* = \{(G_1 - g^{av}), \dots, (G_{G_j} - g^{av})\}$

$a = \arg \max G^*$

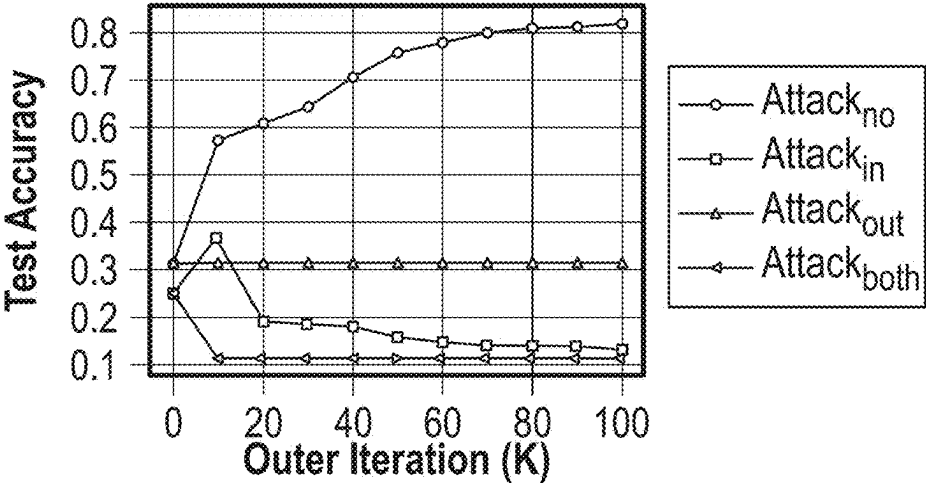
Compute $G = G \setminus G_a$

$j = j + 1$

end while

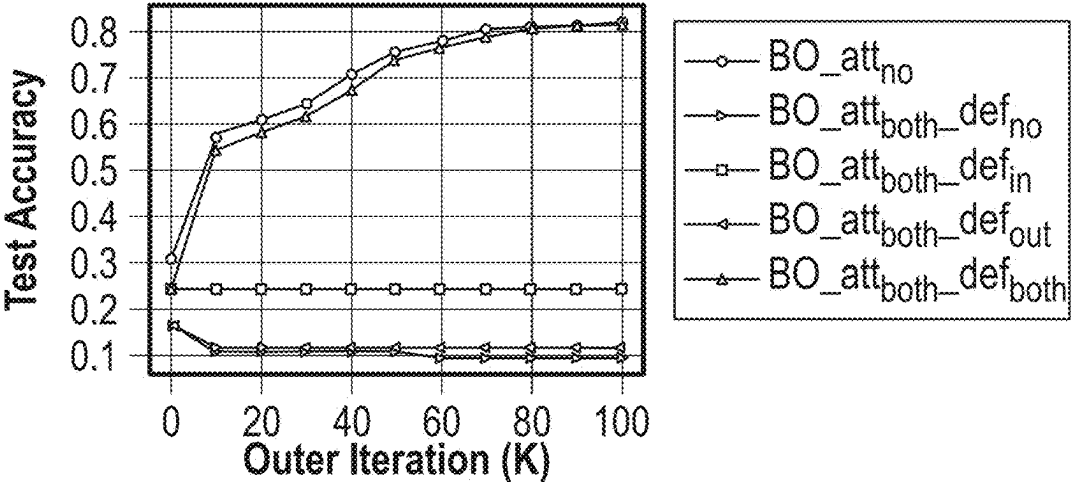
$g^+ = (N - [\eta_x N])^{-1} \sum_{i=1}^{N - [\eta_x N]} G_i$

FIG. 7



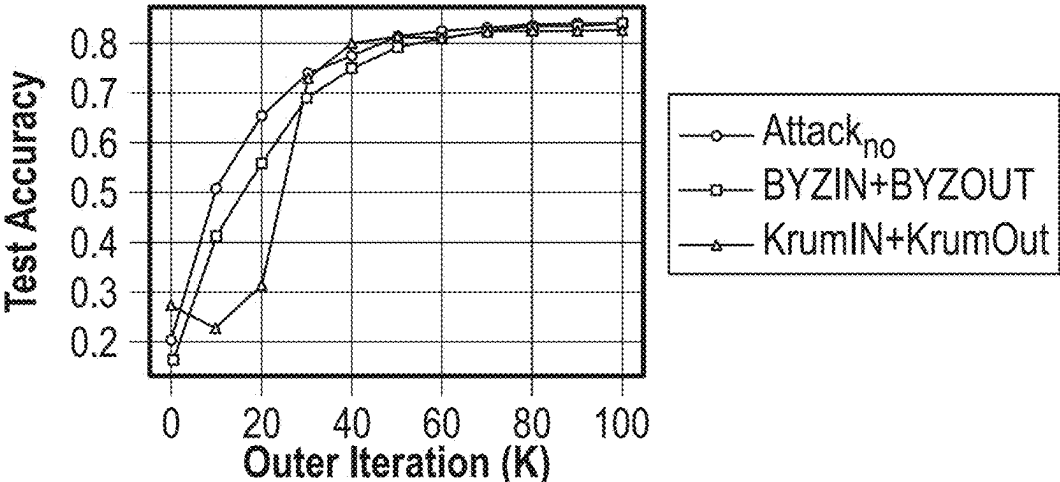
(a) Attacking Different Levels ($B=1$)

FIG. 8A



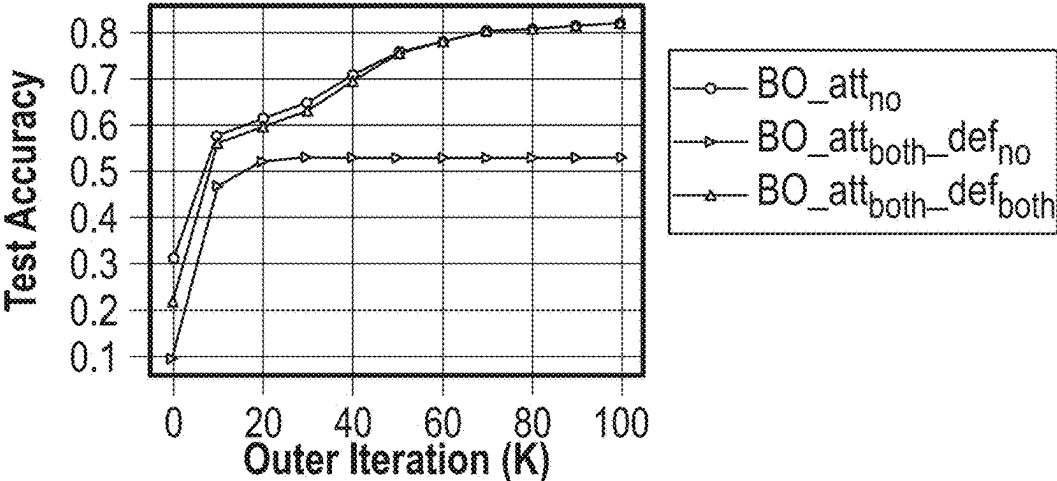
(b) Defending Different Levels ($B=3$)

FIG. 8B



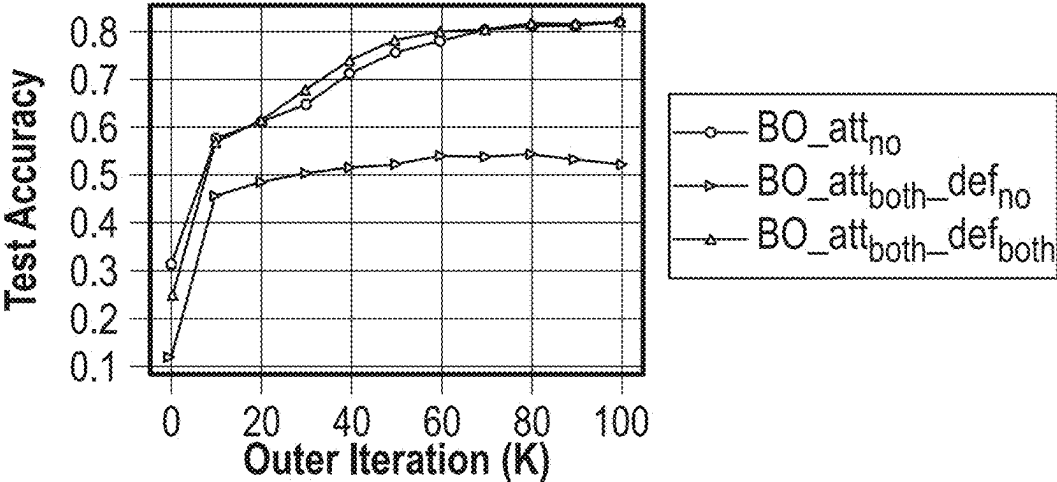
(c) Defense Performance

FIG. 8C



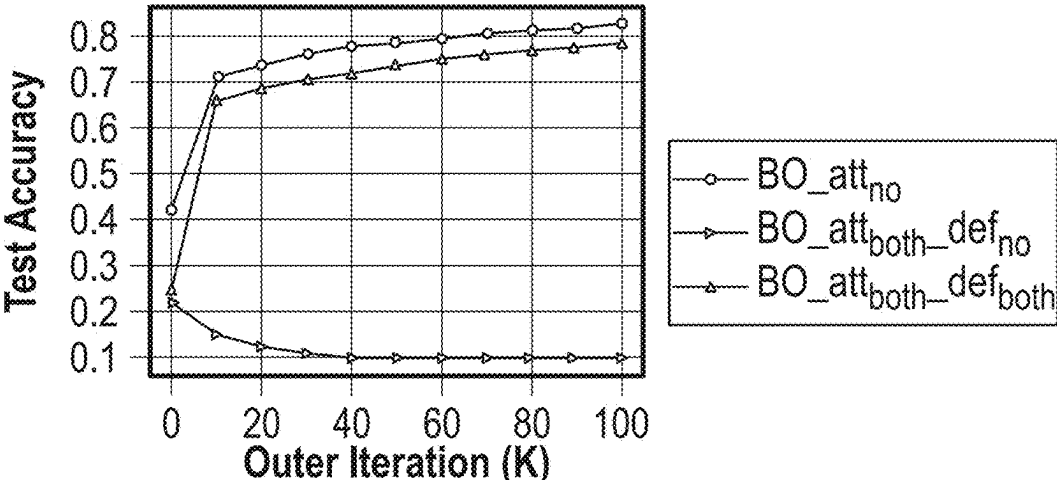
(a) Gaussian Attack

FIG. 9A



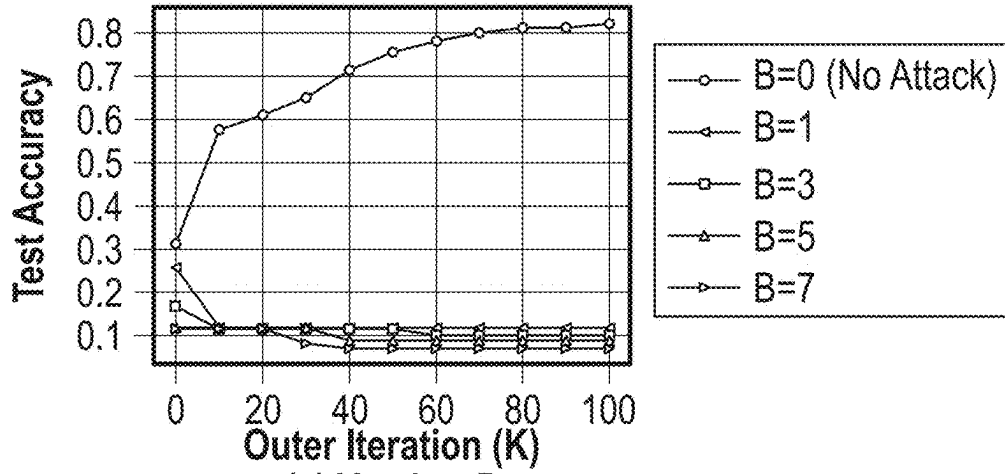
(b) Sign-Flipping Attack

FIG. 9B

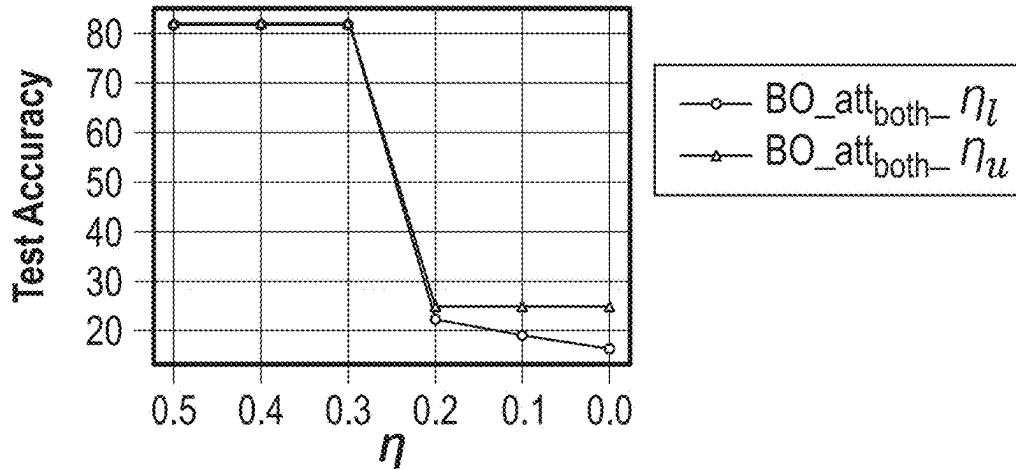


(c) Result on Non-iid Data

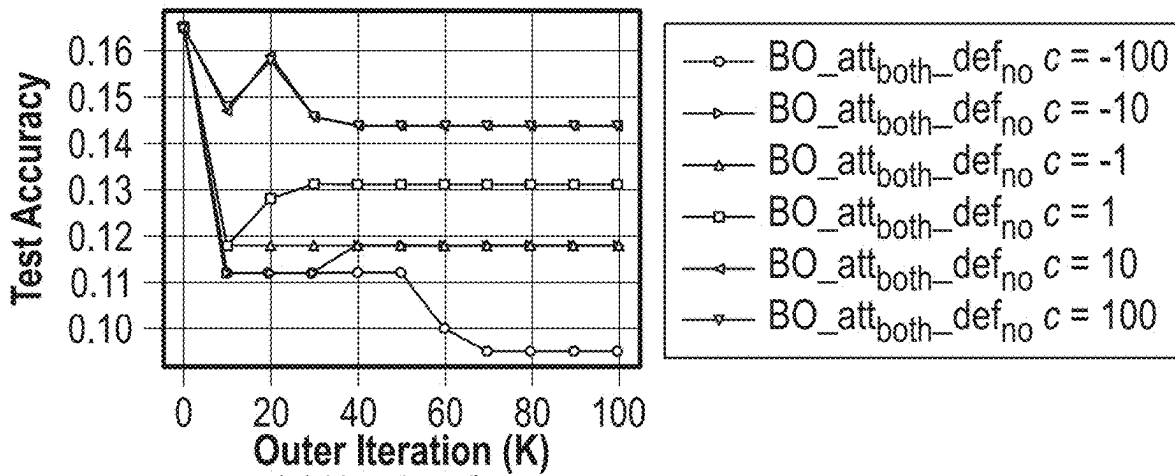
FIG. 9C



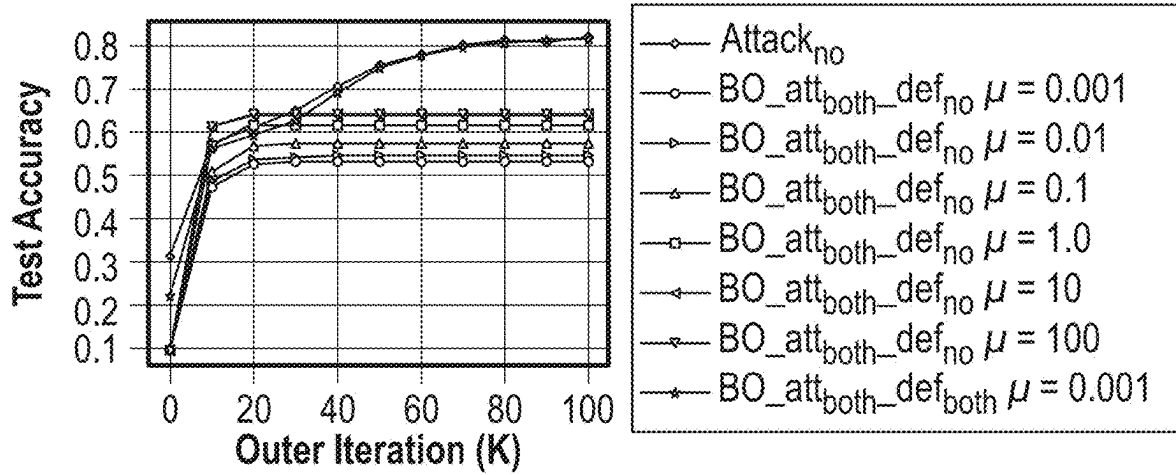
(a) Varying B
FIG. 10A



(b) Varying η_w, η_l
FIG. 10B

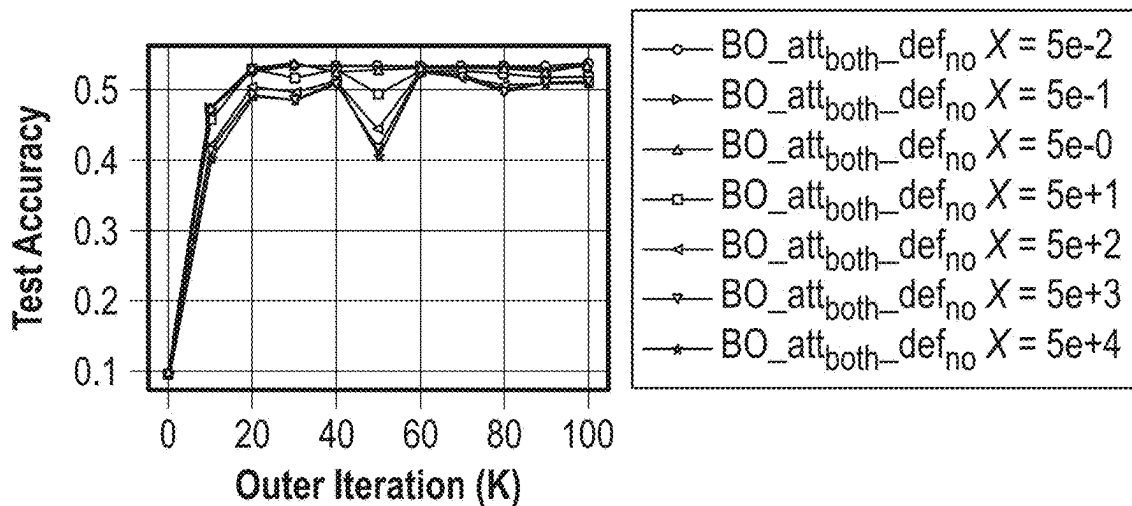


(c) Varying C_{out}
FIG. 10C



Ablation Study for Varying μ_{out}^G

FIG. 11



Ablation Studies for Varying σ_{out}^G

FIG. 12

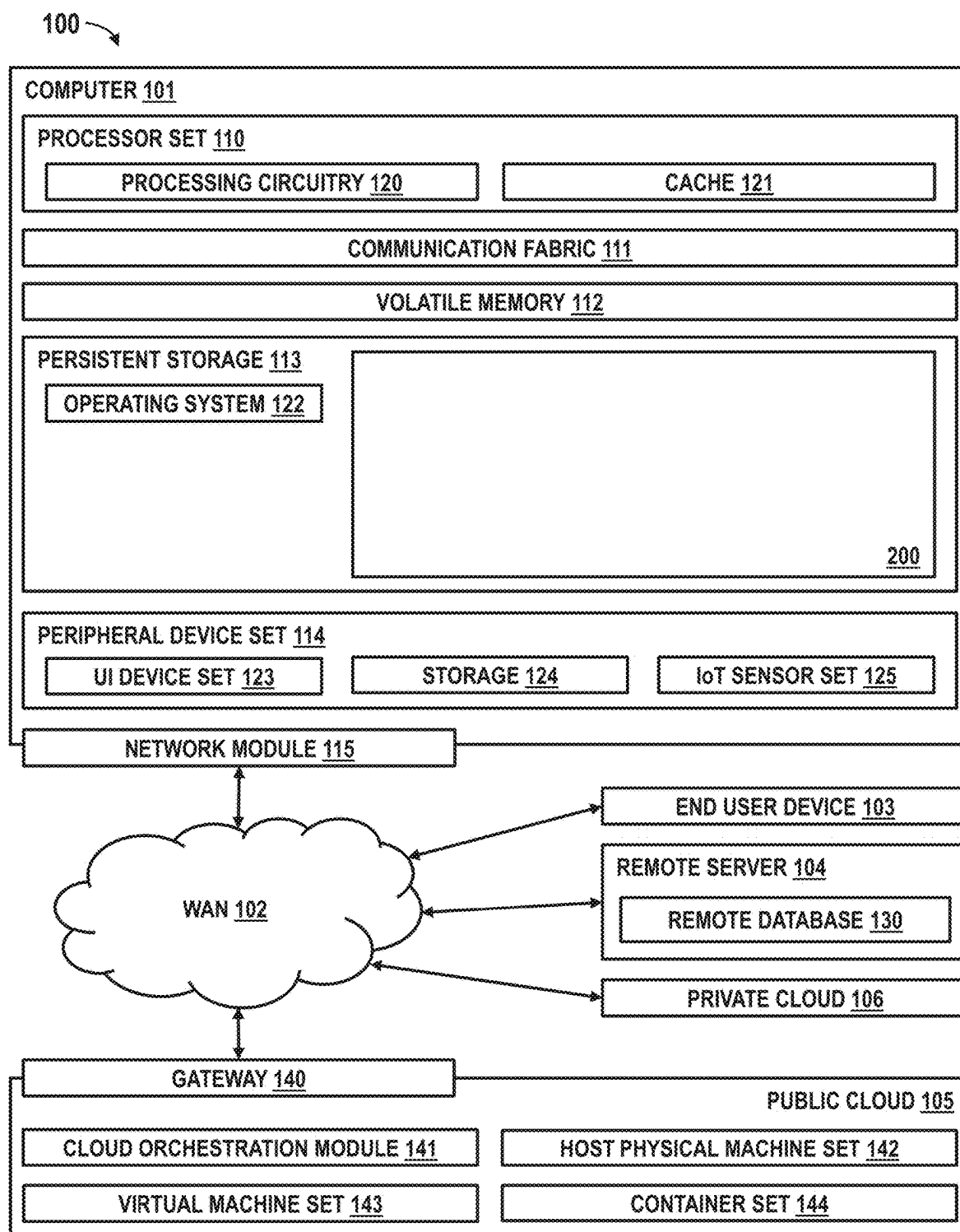


FIG. 13

BYZANTINE-RESILIENT BILEVEL FEDERATED LEARNING

BACKGROUND

[0001] The present invention relates generally to the electrical, electronic and computer arts and, more particularly, to machine learning.

[0002] Federated Learning (FL) is a distributed machine learning (ML) paradigm where a global model is trained from data distributed at multiple client sites, without sharing the raw data of the clients. The global model is trained through an iterative procedure where clients train local models, send their parameters to a server for aggregation, and the server sends the aggregated model parameters to the clients, which train local models from the received aggregated model parameters in the next iteration.

[0003] Byzantine attacks are, however, an important threat to federated learning (FL) systems and may result in poor quality models and slow convergence. In general, a Byzantine attack in federated learning is an attack where at least one participant in the federated learning is a malicious participant who, for example, is attempting to inappropriately obtain information from other participants, impair the performance of the federated learning, and the like. While these attacks have been widely studied in single-level optimization problems, they are not well understood in bilevel optimization.

BRIEF SUMMARY

[0004] Principles of the invention provide systems and techniques for Byzantine-resilient bilevel federated learning. In one aspect, an exemplary method includes the operations of receiving, by an aggregator, updated parameters associated with a lower-level system from a plurality of client computers; generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model by executing federated learning model training and aggregation; generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model by executing the federated learning model training and aggregation; generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model by executing the federated learning model training and aggregation; generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model by executing the federated learning model training and aggregation; and transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

[0005] In one aspect, a computer program product comprises one or more tangible computer-readable storage media and program instructions stored on at least one of the one or more tangible computer-readable storage media, the

program instructions executable by a processor, the program instructions comprising receiving, by an aggregator, updated parameters associated with a lower-level system from a plurality of client computers; generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model by executing federated learning model training and aggregation; generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model by executing the federated learning model training and aggregation; generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model by executing the federated learning model training and aggregation; generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model by executing the federated learning model training and aggregation; and transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

[0006] In one aspect, an apparatus comprises a memory and at least one processor, coupled to the memory, and operative to perform operations comprising receiving, by an aggregator, updated parameters associated with a lower-level system from a plurality of client computers; generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model by executing federated learning model training and aggregation; generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model by executing the federated learning model training and aggregation; generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model by executing the federated learning model training and aggregation; generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model by executing the federated learning model training and aggregation; and transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

[0007] As used herein, “facilitating” an action includes performing the action, making the action easier, helping to carry the action out, or causing the action to be performed. Thus, by way of example and not limitation, instructions executing on a processor might facilitate an action carried out by instructions executing on a remote processor, by

sending appropriate data or commands to cause or aid the action to be performed. Where an actor facilitates an action by other than performing the action, the action is nevertheless performed by some entity or combination of entities.

[0008] Techniques as disclosed herein can provide substantial beneficial technical effects. Some embodiments may not have these potential advantages and these potential advantages are not necessarily required of all embodiments. By way of example only and without limitation, one or more embodiments may provide one or more of:

- [0009] improvements to the technical field of machine learning using a Byzantine-resilient bilevel approach to mitigate Byzantine attacks, even under adaptive attacks where the adversary explicitly targets the bilevel optimization;
 - [0010] efficient defense against Byzantine attacks in the bilevel FL setting by adaptively tuning the performance based on the number of Byzantine clients;
 - [0011] efficient defense against Byzantine attacks in the bilevel FL setting for use cases such as federal hyperparameter optimization; and
 - [0012] exemplary embodiments match the performance of the conventional non-federated, non-Byzantine bilevel methods as well as the conventional federated, non-Byzantine bilevel methods, despite the challenges incurred due to the Byzantine attacks.
- [0013] These and other features and advantages will become apparent from the following detailed description of illustrative embodiments thereof, which is to be read in connection with the accompanying drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

[0014] The following drawings are presented by way of example only and without limitation, wherein like reference numerals (when used) indicate corresponding elements throughout the several views, and wherein:

- [0015] FIG. 1 is a high-level diagram of a federated learning system;
- [0016] FIG. 2 illustrates the operation of federated bilevel optimization under Byzantine attacks;
- [0017] FIG. 3 is an example algorithm (Algorithm 1) for the Byzantine-resilient federated bilevel algorithm (BILANTINE), in accordance with an example embodiment;
- [0018] FIG. 4 is an example subroutine algorithm (Algorithm 2, BYZIN) for a Byzantine-resilient gradient-based optimizer involving local steps for the Byzantine-resilient federated bilevel algorithm, in accordance with an example embodiment;
- [0019] FIG. 5 is an example subroutine algorithm (Algorithm 3, BYZOUT) for a Byzantine-resilient gradient-based optimizer involving local steps for the Byzantine-resilient federated bilevel algorithm, in accordance with an example embodiment;
- [0020] FIG. 6 is an example algorithm for computing the Hessian inverse, in accordance with an example embodiment;
- [0021] FIG. 7 is an example algorithm (Algorithm 5) for the seminal component for both subroutines which provides resilience against Byzantine clients by adaptively removing the outliers in accordance with an example embodiment;
- [0022] FIG. 8A shows the test accuracy vs the number of iterations on the data reweighting task for $M=10$ and $B=1$ under same-value attacks, where M and B denote the num-

ber of honest and Byzantine clients, respectively, in accordance with an example embodiment;

[0023] FIG. 8B shows the test accuracy vs the number of iterations for $M=10$ and $B=3$ under same-value attacks, in accordance with an example embodiment;

[0024] FIG. 8C compares the disclosed method against the no-attack method and a new baseline created for the disclosed bilevel setting based on a conventional single-level defense method, in accordance with an example embodiment;

[0025] FIGS. 9A-9B illustrate the test accuracy performance under Gaussian attack and sign-flipping attack, in accordance with an example embodiment;

[0026] FIG. 9C shows the test accuracy performance on the non-iid (non-independent and identically distributed) setting under same-value attacks, in accordance with an example embodiment;

[0027] FIG. 10A shows the effect of varying the number of Byzantine clients B , in accordance with an example embodiment;

[0028] FIG. 10B shows the effect of varying the assumed proportion of upper-and lower-level Byzantine clients, in accordance with an example embodiment;

[0029] FIG. 10C shows the effect of varying c_{out} , the hyperparameter in the same-value attacks, in accordance with an example embodiment;

[0030] FIGS. 11-12 show the effect of varying the parameters μ_{out}^G and σ_{out}^G of the Gaussian attacks while keeping all other parameters constant, in accordance with an example embodiment;

[0031] FIG. 13 depicts a computing environment according to an embodiment of the present invention.

[0032] It is to be appreciated that elements in the figures are illustrated for simplicity and clarity. Common but well-understood elements that may be useful or necessary in a commercially feasible embodiment may not be shown in order to facilitate a less hindered view of the illustrated embodiments.

DETAILED DESCRIPTION

[0033] Principles of inventions described herein will be in the context of illustrative embodiments. Moreover, it will become apparent to those skilled in the art given the teachings herein that numerous modifications can be made to the embodiments shown that are within the scope of the claims. That is, no limitations with respect to the embodiments shown and described herein are intended or should be inferred.

[0034] FIG. 1 is a high-level diagram of a federated learning system. Federated learning systems generally include an aggregator server 212 and multiple clients 216, 220, 224 who seek to train a global model based on their private local datasets $D_1, D_2, \dots, D_N$. The aggregator server 212 that monitors the federated learning process issues queries to clients 216, 220, 224, collects responses from clients 216, 220, 224 and aggregates the collected responses to update the global model M . (In FIG. 1, Q represents a query issued by the aggregator server 212 to learn a global predictive model M .) Given the current weight (model parameter), gradients and new model parameters are requested. Information about a specific label (class), class counts, and the like are also requested. P represents clients (participants) 216, 220, 224 ($P_1, P_2, \dots, P_N$ where P_i is the i^{th} client that respond to aggregator queries; each client

responds based on its local dataset $D_1, D_2, \dots, D_N$ (generally D_i is the data for the i^{th} client). The responses are denoted $R_1, R_2, \dots, R_N$.

[0035] Federated learning typically assumes a single level of optimization, where the objective is to minimize the training loss of the global model. New machine learning applications (hyperparameter optimization (HPO), data heterogeneity, and the like) require more than just training a model. In this case, bilevel FL is used, where the optimization is split between two hierarchical objectives, with the lower-level optimization specific to the model training, and the upper-level optimization specific to a second objective, such as robustness to distribution shifts, fast adaptation, and automation.

[0036] To tackle emerging learning requirements and challenges, such as robustness and automation, many machine learning problems today involve nested structures, and are thus often formulated as bilevel learning problems. In one example embodiment, bilevel learning in the federated setting and in the presence of Byzantine clients is considered, and a Byzantine-resilient bilevel federated optimization algorithm (referred to as BILANTINE herein) and a corresponding theoretical analysis establishing its convergence rate are disclosed. The performance of an exemplary embodiment was empirically evaluated on a data reweighting task under several attacks to show the efficacy of the exemplary embodiment. It is shown that the exemplary embodiment can nearly achieve the same performance as would be achieved without any attack under several settings.

[0037] Generally, federated learning (FL) is a distributed machine learning (ML) paradigm where a global model is trained from data distributed at multiple client sites **216, 220, 224**, without sharing of the clients' raw data. The global model is trained through an iterative procedure where clients **216, 220, 224** train local models, send their parameters to the aggregator server **212** for aggregation and the aggregator server **212** sends the aggregated model parameters to the clients **216, 220, 224** which train local models from the received aggregated model parameters in the next iteration. Most existing works in FL focus on the single-level problem of training a global model from distributed data. However, emerging FL applications may require ML training tasks with additional learning criteria beyond just training a global model, such as robustness to imbalanced heterogeneous data and distribution shifts, fast adaptation, and automation. Such criteria can be captured with bi-level learning problem formulations where the upper-level solution to the training problem depends on the lower-level solution to the problem that is reflecting the additional learning criteria. Indeed, many ML tasks today admit a bilevel structure including meta-learning, reinforcement learning, hyperparameter optimization, adversarial learning, neural architecture search and continual learning. However, bilevel learning problems have been primarily studied in non-FL settings where all data is available at a central location.

[0038] In example embodiments, the problem of combating Byzantine attacks in bilevel FL is addressed. Byzantine attacks are generally due to clients **216, 220, 224** being compromised by malicious hackers or client hardware/software malfunctions, which inject dubious data into the FL training process and severely affect the resulting global model performance. To overcome this challenge, a series of robust aggregation algorithms have been proposed in the single-level FL setting. The bilevel FL structure, however,

poses new challenges in the discovery of efficient Byzantine-resilient methods and their analysis. These challenges are partially due to the nested (i.e., bilevel) structure of the problem, where the solution of the upper-level problem depends on the solution of the lower-level problem, making it difficult to decipher which approach would optimally defend each level.

[0039] Generally, an efficient method to defend against Byzantine attacks in bilevel FL is disclosed. More specifically, a technique entitled "BILANTINE," the first Byzantine-resilient algorithm for bilevel federated optimization is disclosed. References herein to "BILANTINE" should be understood to refer to one or more exemplary embodiments and it should not necessarily be inferred that features described as present in "BILANTINE" are present in all embodiments of the invention. Exemplary embodiments efficiently defend against Byzantine attacks in the bilevel FL setting by adaptively tuning the performance based on the number of Byzantine clients. In one or more embodiments, this is achieved by two robust aggregation algorithms at each level that interact iteratively by estimating the inner global model and the outer global variable, while adaptively removing Byzantine clients viewed as outliers.

[0040] The convergence properties of BILANTINE are theoretically analyzed below. Such analysis is challenging in the bilevel setting, due to the nested problem formulation and the interaction of the two robust aggregation algorithms. Despite the challenges due to the Byzantine attacks, we have found that the convergence rate of BILANTINE to a near stationary point of (1) is $\mathcal{O}(K^{-0.5})$, matching the convergence rate of the non-FL, non-Byzantine bilevel methods as well as the bilevel FL non-Byzantine methods.

[0041] A comprehensive empirical evaluation is provided to demonstrate the efficacy of BILANTINE to defend against several Byzantine attacks. It is shown that a single Byzantine client **216, 220, 224** can disrupt the bilevel FL training process if no measure is taken and that a robust aggregation algorithm applied only on a single level cannot address Byzantine attacks. Through extensive experiments, including ablation studies that explore the impact of environment, attack, and BILANTINE parameters, it is shown that BILANTINE is robust in mitigating the attacks with the global FL model and often achieves comparable performance to the model trained when no Byzantine attack is present.

Byzantine-Resilient FL

[0042] In the deep learning literature, a Byzantine resilient algorithm to defend distributed implementations of stochastic gradient descent (SGD) has been proposed. However, most of the conventional works have a common problem: they lose a lot of useful information to retain the convergence of the training algorithm. For example, one conventional approach selects only one out of all the uploaded gradients. One conventional approach is tailored to single-level FL problems. Moreover, in the data heterogeneous (non-independent and identically distributed (iid)) setting, finding the average of the honest clients' gradients is more challenging. All these approaches were proposed to defend against Byzantine attacks in the single-level FL setting.

Centralized Bilevel Learning

[0043] The nested problem of bilevel optimization can be traced back to conventional works. Thereafter, various

efforts have been made to solve the bilevel optimization problem. Some earlier works reduce the bilevel problem to a single-level problem. However, reduction to a single-level problem poses several challenges owing to a large number of constraints. Recently, gradient-based first-order methods have been proposed. Although these methods perform well empirically, they either lack theoretical guarantees or focus only on asymptotic performance analysis. Moreover, conventional approaches provide non-asymptotic analysis of nonconvex SGD-type methods for bilevel problems. However, all these works focus on the non-federated bilevel setting.

Distributed Bilevel Learning

[0044] Building on centralized bilevel learning, the bilevel problem was formulated in the FL setting where multiple clients seek to learn lower and upper models under a central server. One conventional approach employs the non-robust mean aggregation rule where the global model is computed by taking the uniform average of the local models and such an aggregation poses a major security challenge in FL via Byzantine attacks. Moreover, decentralized bilevel optimization approaches that perform learning without a central server are not targeted to the client-server FL setting and none of them are robust to Byzantine attacks.

Federated Bilevel Learning Formulation

[0045] The set of clients $M=\{1, \dots, M\}$ is such that $M \subseteq N$ where N denotes the total set of clients, where $N=\{1, \dots, N\}$. One aim is to solve the following federated bilevel optimization problem:

$$\min_{x \in \mathbb{R}^d} f(x) := \frac{1}{M} \sum_{i=1}^M f_i(x, y^*(x)) \quad (\text{upper}) \quad (1a)$$

$$\text{s.t. } y^*(x) = \arg \min_{y \in \mathbb{R}^{d'}} \frac{1}{M} \sum_{i=1}^M g_i(x, y) \quad (\text{lower}) \quad (1b)$$

[0046] In the optimization literature, one stochastic bilevel problem (see equations 1a and 1b) is referred to as the stochastic bilevel problem, where the upper-level optimization problem depends on the solution of the lower-level optimization problem over $y \in \mathbb{R}^{d'}$, denoted as $y^*(x)$, which depends on the value of the upper-level variable $x \in \mathbb{R}^d$. Each client $i \in M=\{1, \dots, M\}$ has its own upper and lower functions $f_i = \mathbb{E}_{\xi_i} [f_i(x, y^*(x); \xi_i)]$ and $g_i = \mathbb{E}_{\zeta_i} [g_i(x, y(x); \zeta_i)]$, where $(\xi_i, \zeta_i) \sim (C_i, D_i)$ denote the upper and lower data distributions. In practical scenarios, the dataset may not be i.i.d. across clients and, therefore, (C_i, D_i) can be different from (C_j, D_j) for $i \neq j$.

A General Bilevel FL Method

[0047] A general approach to solve to the stochastic bilevel problem (equations 1a and 1b) is via a conventional iterative algorithm operating in two nested loops. With the detailed updates specified in the section entitled “Byzantine-resilient Federated Bilevel Learning Algorithm,” the inner loop approximates y^* while the outer loop operates approximates x^* .

Inner Loop

[0048] Each client i receives the current global model x^k and $y^{k,t}$ from the aggregator server **212** and obtains an

estimate of its local model $y_i^{k,t+1}$. The aggregator server **212** receives the local models from the clients and updates the model as:

$$y^{k,t+1} = \frac{1}{M} \sum_{i=1}^M y_i^{k,t+1} \quad (2)$$

Outer Loop

[0049] In the outer loop, each client i receives the current model x^k and $y^{k,t}$ from the aggregator server **212** and obtains an estimate of its local model x_i^{k+1} . The aggregator server **212** receives the local models from the clients **216, 220, 224** and updates the model as:

$$x^{k+1} = \frac{1}{M} \sum_{i=1}^M x_i^{k+1} \quad (3)$$

Negative Effects of Byzantine Clients

[0050] FIG. 2 illustrates the operation of federated bilevel optimization under Byzantine attacks. In the federated setup, Byzantine failures may occur when clients **216, 220, 224** upload their models after being under attack by hackers or after hardware/software malfunctions. Consequently, these Byzantine clients **216, 220, 224** upload arbitrary information to the aggregator server **212**, significantly deteriorating the learning process. Let $\hat{x}_i^{k+1}$ and $\hat{y}_i^{k,t+1}$ denote the upper/lower models uploaded by client i . A generalized Byzantine model is given as:

$$\hat{x}_i^{k+1} = \begin{cases} x_i^{k+1} & \text{if } i \in M \\ * & \text{if } i \in B \end{cases} \quad (4)$$

$$\hat{y}_i^{k,t+1} = \begin{cases} y_i^{k,t+1} & \text{if } i \in M \\ * & \text{if } i \in B \end{cases} \quad (5)$$

[0051] where $B \subseteq \mathcal{N}$ denotes the set of Byzantine clients **216, 220, 224**, $M \subseteq \mathcal{N} \setminus B$ denotes the set of honest clients **216, 220, 224** and $*$ denotes an arbitrary vector. B is denoted as $|B|$ and M as $|M|$, where $M+B=N$. For example, if a client i is Byzantine, it can choose to send $\hat{x}_i^{k+1} = \infty$ or $\hat{y}_i^{k,t+1} = \infty$. It can also send $\hat{x}_i^{k+1} = -\sum_{i=1}^{N-1} x_i^{k+1}$ or $\hat{y}_i^{k,t+1} = -\sum_{i=1}^{N-1} y_i^{k,t+1}$. Under these cases, the global aggregated models x_{k+1} or $y_{k,t+1}$ at the aggregator server **212** will become either infinite or null. Indeed, it is shown that even a single Byzantine client **216, 220, 224** in either level of the optimization problem (1) can disrupt the training process (see FIG. 8A).

Byzantine-Resilient Federated Bilevel Learning Algorithm

[0052] FIG. 3 is an example algorithm (Algorithm 1) for the Byzantine-resilient federated bilevel algorithm (BILANTINE), in accordance with an example embodiment. Formally summarized in Algorithm 1, BILANTINE operates in two nested loops. The outer loop operates in rounds $k \in \{1, \dots, K\}$. Within each round k , the inner loop operates in iterations $t \in \{1, \dots, T\}$. Using x^k and y^k , the inner iteration t estimates a global model $y^{k,t+1}$ as the inner optimization

variable $y^{*(x^k)}$ as the output of BYZIN. The final output $y^{k+1}=y^{k,T}$ of the inner loop is then input to BYZOUT to estimate the upper global variable x^{k+1} .

[0053] FIG. 4 is an example subroutine algorithm (Algorithm 2, BYZIN) for a Byzantine-resilient gradient-based optimizer involving local steps $v \in \{0, \dots, \tau_i-1\}$ for the Byzantine-resilient federated bilevel algorithm, in accordance with an example embodiment. FIG. 5 is an example subroutine algorithm (Algorithm 3, BYZOUT) for a Byzantine-resilient gradient-based optimizer involving local steps $v \in \{0, \dots, \tau_i-1\}$ for the Byzantine-resilient federated bilevel algorithm, in accordance with an example embodiment. BYZIN performs the local steps on the inner variable $Y_{i,v}$ while BYZOUT performs local steps on the upper variable $x_{i,v}$. FIG. 7 is an example algorithm (Algorithm 5) for the seminal component for both subroutines (Algorithms 2 and 3) which provides resilience against Byzantine clients by adaptively removing the outliers in accordance with an example embodiment.

[0054] Solving the stochastic bilevel problem (equations 1a and 1b) requires computing the gradient of the upper-level function

$$\left(\text{i.e. } \nabla f(x) = \frac{1}{M} \sum_{i=1}^M f_i(x) \right) \quad (5)$$

usually referred to as the hypergradient. The Hessian Inverse (InvHG) sub-routine is leveraged to efficiently approximate $\nabla^2 f(x)$.

Hessian Inverse (InvHG)

[0055] FIG. 6 is an example algorithm for computing the Hessian inverse, in accordance with an example embodiment. The computation of the hypergradient is given by:

$$\nabla f_i(x, y^*(x)) = \nabla_x f_i(x, y^*(x)) - \nabla_{xy}^2 g(x, y^*(x)) [\nabla_y^2 g(x, y^*(x))]^{-1} \nabla_y f_i(x, y^*(x)) \quad (6)$$

[0056] Due to the nested nature of the problem, the upper-level problem depends on the solution of the lower-level problem $y^*(x)$. Moreover, the computation of Hessian inverse in (6) requires the global Hessian (which is unavailable at the client **216**, **220**, **224**). The InvHG subroutine is used to address this challenge and compute the Hessian inverse. InvHG uses a client sampling strategy to efficiently approximate the second term on the right-hand side (RHS) of (6). More specifically, given $R \in \mathbb{N}$, $R' \in \{0, \dots, R-1\}$ and $\mathcal{N}_0, \dots, \mathcal{N}_R \in \mathcal{N}$ uniformly at random (UAR) are chosen. For $i \in \mathcal{N}$:

$$h_i(x, y) = -\nabla_{xy}^2 g_i(x, y; \xi_i) p_{R'} \quad (7)$$

where

$$p_{R'} = |\mathcal{N}_0|^{-1} \hat{H}_y \sum_{i \in \mathcal{N}_0} \nabla_y f_i(x, y, \xi_{i,0})$$

-continued
and $\hat{H}_y$ is

$$\hat{H}_y = \frac{R}{\ell_{g,1}} \prod_{r=1}^{R'} \left(I - \frac{1}{\ell_{g,1} |\mathcal{N}_r|} \sum_{i=1}^{|\mathcal{N}_r|} \nabla_y^2 g_i(x, y; \xi_{i,r}) \right). \quad (8)$$

BYZOUT

[0057] For BYZOUT, the notation is simplified by denoting input of the function as (x, y^*) instead of (x^k, y^{k+1}) and the output as x^+ instead of x^{k+1} . All clients **216**, **220**, **224** in parallel approximate their indirect gradients h_i . The aggregator server **212** then computes the global $\bar{h}$ via RES that eliminates the outliers and takes the average of the remaining h_i . The aggregator server **212** transmits $\bar{h}$ to all the clients **216**, **220**, **224**. Each client **216**, **220**, **224** then uses and performs τ_i local updates on the upper variable x_i and communicates x_{i,τ_i} to the aggregator server **212**. Finally, the aggregator server **212** calls the subroutine RES to eliminate the Byzantine models.

BYZIN

[0058] For BYZIN, the notation is simplified by denoting input of the function as (x, y) instead of $(x^k, y^{k,r})$. In each inner iteration t , each client **216**, **220**, **224** locally updates its model y_i multiple times before transmitting it to the aggregator server **212**. The aggregator server **212** filters out the outliers using RES before performing the aggregation step.

RES

[0059] The design of the disclosed RES (Algorithm 5) is based on the observation that the honest models are not very dissimilar and that the Byzantine models must be sufficiently different from the honest models in order to successfully alter the aggregation results. And if Byzantine models are similar to the honest models, the attack will cause no harm to the aggregation results. At each iteration j , RES allows the aggregator server **212** to eliminate the outliers from the current mean, i.e. it adaptively removes $[\eta_x N]$ models based on the distance between current average g^{av} and the remaining set of $\mathcal{G}$.

Convergence and Robustness Analysis

[0060] The convergence analysis of the proposed BILANTINE algorithm and the impact of Byzantine clients **216**, **220**, **224** on the convergence rate is described below. The following assumption is made on the number of Byzantine clients **216**, **220**, **224** in the subsequent analysis.

Assumption 1 (Proportion of Byzantine Clients)

[0061] With the total number of clients **216**, **220**, **224** denoted as N , it is assumed that no more than half of the upper and lower clients **216**, **220**, **224** are Byzantine, i.e. $B = \eta_u N = \eta_l N$ where $\eta_u, \eta_l < 0.5$. The above assumption is commonly used in single-level Byzantine-resilient approaches.

[0062] The following two assumptions on the upper-and lower-level objective functions are made.

Assumption 2 (Lipschitz Continuity)

[0063] Denote $z = (x, y)$. Assume that $f_i(z)$, $\nabla f_i(z)$, $\nabla g_i(z)$, $\nabla^2 g_i(z)$ are $\ell_{f,0}$, $\ell_{f,1}$, $\ell_{g,1}$, $\ell_{g,2}$ Lipschitz continuous, respectively.

Assumption 3 (Strongly Convex Lower-Level Objective)

[0064] For any fixed x , $g_i(x, y)$ is μ_g -strongly convex in y , that is, $\nabla_{yy}^2 g_i(x, y) \succeq \mu_g I$.

[0065] The following assumption on the stochastic estimates of the derivatives of upper- and lower-level functions is made.

Assumption 4 (Stochastic Derivatives)

[0066] The derivatives $\nabla f_i(z; \xi)$, $\nabla g_i(z; \xi)$, $\nabla^2 g_i(z; \xi)$ are unbiased estimators of $\nabla f_i(z)$, $\nabla g_i(z)$, $\nabla^2 g_i(z)$, respectively, $\forall i \in M$; and their variances are bounded, i.e.

$$\mathbb{E}_\xi [\|\nabla^2 f_i(z; \xi) - \nabla^2 f_i(z)\|^2] \leq \sigma_f^2$$

$$\mathbb{E}_\xi [\|\nabla^2 g_i(z; \xi) - \nabla^2 g_i(z)\|^2] \leq \sigma_{g,1}^2$$

$$\mathbb{E}_\xi [\|\nabla^2 g_i(z; \xi) - \nabla^2 g_i(z)\|^2] \leq \sigma_{g,2}^2$$

for some σ_f^2 , $\sigma_{g,1}^2$, $\sigma_{g,2}^2$.

[0067] Assumptions 2 and 3 together ensure that the first- and second-order derivatives of the objectives f and g and the solution mapping $y^*(x)$ are well-behaved. If Assumption 3 does not hold, the analysis of bilevel problems becomes intractable as the Hessian in (6) is not invertible. This assumption is standard in bilevel optimization and, as described in the introduction, it has not prevented its usage in various application domains. Exemplary embodiments do not necessarily make any assumption on the objective of the upper-level problem, which can be non-convex. Assumption 4 is the counterpart of the unbiasedness and bounded variance assumption used in the single-level stochastic optimization literature. All the above assumptions are standard in the non-asymptotic analysis of stochastic bilevel optimization literature.

[0068] Next, it is shown that the results of the RES aggregation are close to the average of the honest models. Specifically, the following lemma is provided. For notational brevity, the iteration indexes are omitted.

Lemma 1

[0069] Denote upper and lower honest models (i.e., local models of honest clients **216**, **220**, **224**)

[0070] as $x_1, x_2, \dots, x_M$ and $y_1, y_2, \dots, y_M$, respectively. Further, denote upper and lower Byzantine models as $a_1, a_2, \dots, a_B$ and $c_1, c_2, \dots, c_B$, respectively; at each level, $M+B=N$. Let

$$x = \frac{1}{M} \sum_{j=1}^M x_j$$

and

$$y = \frac{1}{M} \sum_{j=1}^M y_j.$$

If it is assumed that $\exists \epsilon_u > 0$, $\|x_f - x\| < \epsilon_u$ and $\exists \epsilon_l > 0$, $\|y_f - y\| < \epsilon_l$ for $j \in \{1, 2, \dots, M\}$, then after the process of RES, it follows:

$$\|\bar{x} - x\|^2 \leq \frac{\epsilon_u^2}{(1 - 2\eta_u)^2}$$

and

$$\|\bar{y} - y\|^2 \leq \frac{\epsilon_l^2}{(1 - 2\eta_l)^2}$$

where

$$\bar{x} = \frac{1}{p_u} \sum_{i=1}^{p_u} x_i;$$

$$\bar{y} = \frac{1}{p_l} \sum_{i=1}^{p_l} y_i;$$

$$p_u = N - \lceil \eta_u N \rceil$$

and

$$p_l = N - \lceil \eta_l N \rceil;$$

η_u and η_l denote the assumed proportion of upper and lower Byzantine clients.

[0071] Lemma 1 bounds the distance between the average of the honest variables and the remaining variables.

[0072] Next, the convergence rate of BILANTINE is characterized by using a new Lyapunov function defined as:

$$\mathbb{W}^k := f(\bar{x}^k) + \frac{M_f}{L_y} (\|y^*(x^k) - y^k\|^2) \quad (9)$$

where M_f and L_y are constants.

[0073] The above Lyapunov function was designed to be specifically tailored to show the convergence BILANTINE. This Lyapunov function is slightly different from the one used by conventional techniques which proved the convergence either under bilevel non-FL setting or bilevel FL setting under no Byzantine attacks; e.g., they use $f(x^k)$ as the first term on the right-hand side (RHS) of Equation (9).

Theorem 1

[0074] Under Assumptions 2-4, define constants as $\alpha_i^k = \alpha_k / \tau_i$, $\beta_i^k = \beta_k / \tau_i$, $\forall i \in \mathcal{N}$, where

$$\beta_k = \frac{\bar{\beta} \alpha_k}{T}, \alpha_k = \min \left\{ \bar{\alpha}_1, \bar{\alpha}_2, \frac{\bar{\alpha}}{\sqrt{K}} \right\} \quad (10)$$

and where $\bar{\alpha}_1$, $\bar{\alpha}_2$, $\bar{\alpha}$, and $\bar{\beta}$ are some positive constants independent of K . Then for $T=1$ and $\tau_i=1$, the iterates $\{(x_k, y_k)\}_{k \geq 0}$ generated by BILANTINE satisfy

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} [\|\nabla f(x^k)\|^2] = O \left(\frac{1}{\min\{\bar{\alpha}_1, \bar{\alpha}_2\} K} + \frac{\bar{\alpha} \max(\sigma_{g,1}^2, \sigma_{g,2}^2, \sigma_f^2)}{\sqrt{K}} + b^2 \right) + \quad (11)$$

$$O \left(\frac{\frac{\epsilon_u^2}{(1-2\eta_u)^2}}{\min\{\bar{\alpha}_1, \bar{\alpha}_2\} K} + \frac{\frac{\epsilon_u^2}{(1-2\eta_u)^2}}{\bar{\alpha} \sqrt{K}} + \frac{\epsilon_u^2}{(1-2\eta_u)^2} + \frac{\epsilon_l^2}{(1-2\eta_l)^2} \right)$$

-continued
where

$$b = \kappa_g \ell_{f,1} ((\kappa_g - 1) / \kappa_g)^R, \kappa_g = \ell_{g,1} / \mu_g$$

is the condition number of $g(x, \cdot)$, and R is the input of InvHG. (Theorem 1 implies that the convergence rate of BILANTINE to a near stationary point of (1) is $\mathcal{O}(K^{-0.5})$, matching the convergence rate of the conventional non-federated non-Byzantine bilevel methods as well as the federated non-Byzantine bilevel methods, despite the challenges due to the Byzantine attacks).

Impact of Byzantine Clients

[0075] Compared to a conventional technique, BILANTINE incurs an additional asymptotic error term:

$$\lim_{K, R \rightarrow \infty} \frac{1}{K} \sum_{k=1}^K \mathbb{E}[\|\nabla f(x^k)\|^2] = \mathcal{O}\left(\frac{\epsilon_u^2}{(1-2\eta_u)^2} + \frac{\epsilon_g^2}{(1-2\eta_l)^2}\right). \quad (12)$$

This error depends on the proportion of upper and lower Byzantine clients η_u and η_l and the bounds due to data heterogeneity ϵ_u and ϵ_g . Higher (η_u, η_l) yield a rapid, hyperbolic increase of the asymptotic error. Such a term also appears in the single-level Byzantine-resilient FL with non-i.i.d. data. However, this is a worst case bound and the exemplary experiments show that this error is small and yields more graceful performance degradation as (η_u, η_l) increase.

Numerical Evaluation

[0076] Experiments on the disclosed Byzantine-resilient bilevel methods implemented on an open source machine learning framework on a conventional graphics processing unit (GPU) are presented below.

Experimental Setup

[0077] Experiments were conducted on the task of data reweighting for noisy samples on a conventional large database of handwritten digits following the settings of a conventional system. The goal of data reweighting is to train a classifier in the situation that a proportion of the training labels are corrupted. Clean data was used as validation data, the rest as the training data, and one hyperparameter (i.e., weights) was assigned to each training example. More specifically, denote x as the weights of the training data and y as the parameters of the classifier. Let l be the cross-entropy loss corresponding to y in both training objective g and validation objective f . For the training phase, a hyperparameter vector $x \in [0, 1]$ was additionally added, i.e. a reweighted variant of the loss, denoted by

$$g(x, y) = \frac{1}{n} \sum_{j=1}^n \sigma(x^j) l(y, \zeta^j),$$

is defined as the empirical loss on the training set, where $\sigma(\cdot)$ is the sigmoid function and x^j is the weight corresponding to sample ζ^j ; n denotes the number of training samples. The aim is to fit a model y to minimize the training loss and the hyperparameters x to minimize the validation loss.

[0078] 2000, 200, and 1000 images were randomly selected from the conventional large database of handwritten digits for training, validation, and testing, respectively. Each label in the training data is replaced by a wrong label sampled uniformly with a probability of 0.5. x denotes the logits of the weights of the training samples, and y denotes parameters of a multilayer perceptron (MLP) of size $784 \rightarrow 256 \rightarrow 10$. A batch size of 100 was used, and the outer and inner learning rates were set as 10 and 0.3 respectively.

Attack and Defense on Different Levels

[0079] Experiments were conducted on three Byzantine attacks: Gaussian, sign-flipping, and same-value. For the Gaussian attack, each Byzantine client **216, 220, 224** crafts its model from a Gaussian distribution $\sim \mathcal{N}_G(\mu^G, \sigma^G)$. For sign-flipping attacks, each Byzantine client **216, 220, 224** flips the sign of its original model parameters and enlarges the magnitude, i.e. each Byzantine client i calculates the true model $\tilde{x}_i^k$ and then sends $x_i^k = a \tilde{x}_i^k$ to the aggregator server **212**, where a is a negative constant. For same-value attacks, the message sent by each Byzantine client **216, 220, 224** is $c \mathbb{1}$, where $\mathbb{1}$ is an all-one vector and c is a constant. In the following, subscripts $\{a_{in}, a_{out}, c_{in}, c_{out}\}$ are used to denote lower and upper level hyperparameters.

[0080] FIG. 8A shows the test accuracy vs the number of iterations on the data reweighting task for $M=10$ and $B=1$ under same-value attacks, where M and B denote the number of honest and Byzantine clients **216, 220, 224**, respectively, in accordance with an example embodiment. Unless otherwise stated, settings $c_{out}=-100$, $c_{in}=0.001$ are used for same-value attacks. These values are chosen to maximize the negative effect due to Byzantine clients **216, 220, 224**. FIG. 8A illustrates that Byzantine attacks are lethal to bilevel tasks in the presence of even a single Byzantine client **216, 220, 224** at either level of the optimization problem (1). One or more embodiments advantageously provide Byzantine-robust algorithms in the bilevel learning setting to address the problem illustrated in FIG. 8A. FIG. 8B shows the test accuracy vs the number of iterations for $M=10$ and $B=3$ under same-value attacks, in accordance with an example embodiment. The results demonstrate the efficacy of the disclosed BYZIN and BYZOUT methods. Label $BO_att_{\{ \}_def_{\{ \}}}$ represents attack and defense under different settings (e.g. $BO_att_{both_def_{out}}$ refers to the case where there are attacks on both levels but defend only the upper level). It was observed that defense at a single level (lower or upper) is not able to defend against Byzantine attacks; bilevel tasks, therefore, require well-designed defense algorithms at both levels. Second, the disclosed BYZIN and BYZOUT algorithms together can achieve nearly no-attack performance. FIG. 8C compares an exemplary embodiment against the no-attack method and a new baseline created for the disclosed bilevel setting based on a conventional single-level defense method, in accordance with an example embodiment. The exemplary embodiment performs better than the conventional single-level defense method, justifying the disclosed choice of using per-level defense in the bilevel FL setting. Advantageously, some embodiments use known techniques for fast aggregation against Byzantine attacks,

Performance on Non-iid Data

[0081] Considering the data heterogeneity across clients **216, 220, 224**, an exemplary embodiment was also tested on

non-iid data for $M=5$ and $B=3$. To generate the non-iid setting, labels were randomly distributed from two random classes to each honest client **216**, **220**, **224**, i.e. each honest client **216**, **220**, **224** receives data from only two distinct labels. FIG. 9C shows the test accuracy performance on the non-iid setting under same-value attacks. It was observed that the disclosed method performs comparable to the no-attack case, demonstrating that the exemplary embodiment can successfully defend against Byzantine attacks even in the non-iid setting.

Ablation Studies

[0082] Several ablation studies were performed on additional attacks and various pertinent parameters pertaining to the disclosed defense and different attacks. For all the ablation studies, unless otherwise specified, $M=10$ and $B=3$.

Additional Attacks

[0083] FIGS. 9A-9B illustrate the test accuracy performance under Gaussian attack and sign-flipping attack, in accordance with an example embodiment. For the sign-flipping attack, $a_{in}=-0.001$, $a_{out}=-4$. For Gaussian attacks, $\mu_{in}=0.01$, $\sigma_{in}=0.003$, $\mu_{out}=0.001$, and $\sigma_{out}=0.0005$. It was observed that the exemplary embodiment achieves nearly no-attack performance on both attacks.

Varying B

[0084] FIG. 10A shows the effect of varying the number of Byzantine clients B , in accordance with an example embodiment. It was observed that even a single Byzantine client can successfully disrupt the bilevel FL training. Increasing the number of Byzantine clients **216**, **220**, **224** will make the attack more severe, further deteriorating accuracy.

Varying η_u , η_l

[0085] FIG. 10B shows the effect of varying the assumed proportion of upper- and lower-level Byzantine clients **216**, **220**, **224**, in accordance with an example embodiment. For the experiment on η_u , η_l was set to 0.3. Similarly, for the experiment on η_l , η_u was set to 0.3. It was observed that decreasing both η_u and η_l deteriorates the performance. Performance degrades significantly when η attains the value of 0.2. This is because, in the disclosed experimental setting, about 23% of both upper- and lower-level clients are Byzantine (since $M=10$ and $B=3$). It was also observed that decreasing η_l deteriorates the performance by a larger margin. For example, when η is 0.0 (i.e., under no-defense setting), η_u achieves 16.5% test accuracy as compared to 24.7% of η_l .

Varying c_{out}

[0086] FIG. 10C shows the effect of varying c_{out} , the hyperparameter in the same-value attacks (see section entitled “Experimental Setup”), in accordance with an example embodiment. It was observed that a larger negative value achieves a more lethal attack, significantly reducing the testing accuracy. For example, $c_{out}=-100$ achieves a testing accuracy of 9.7% as compared to 14.4% achieved by $c_{out}=-10$. However, from FIG. 8B, even for $c_{out}=-100$, the exemplary embodiment can successfully defend against the attack and achieve nearly no-attack performance.

Varying μ_{out}^G , σ_{out}^G

[0087] FIGS. 11-12 show the effect of varying the parameters μ_{out}^G and σ_{out}^G of the Gaussian attacks while keeping all other parameters constant, in accordance with an example embodiment. It was observed that a smaller value of μ_{out}^G achieves a lower test accuracy, thereby generating a more lethal attack as compared to a larger value. For example, $\mu_{out}^G=0.001$ achieves a test accuracy of 53.27% as compared to 64.15% achieved by $\mu_{out}^G=100$. However, even for $\mu_{out}^G=-0.001$, the exemplary embodiment can successfully defend against the attack and achieve nearly no-attack performance. On the contrary, a larger value of σ_{out}^G achieves a lower test accuracy. For example, $\sigma_{out}^G=5e+4$ achieves a test accuracy of 51.18% as compared to 52.25% achieved by $\sigma_{out}^G=5e-2$. However, from FIG. 9A, it was observed that even for $\sigma_{out}^G=-0.001$, the exemplary embodiment can successfully defend against the attack and achieve nearly no-attack performance.

[0088] Given the discussion thus far, it will be appreciated that, in general terms, an exemplary method, according to an aspect of the invention, includes the operations of receiving, by an aggregator, updated parameters associated with a lower-level system from a plurality of client computers; generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model by executing federated learning model training and aggregation; generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model by executing the federated learning model training and aggregation; generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model by executing the federated learning model training and aggregation; generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model by executing the federated learning model training and aggregation; and transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

[0089] In one example embodiment, the intermediate global lower-level machine learning model is transmitted, by the aggregator, to at least one of the plurality of client computers; revised parameters associated with the lower-level system are received, by the aggregator, from at least one of the plurality of client computers; the global lower-level machine learning model is transmitted, by the aggregator, to at least one of the plurality of client computers; updated parameters associated with the upper-level system are received, by the aggregator, from at least one of the plurality of client computers; the intermediate global upper-level machine learning model is transmitted, by the aggregator, to at least one of the plurality of client computers; and revised parameters associated with the upper-level system are received, by the aggregator, from at least one of the plurality of client computers.

[0090] In one example embodiment, the updated parameters associated with the lower-level system, the revised parameters associated with the lower-level system, the updated parameters associated with the upper-level system and the revised parameters associated with the upper-level system include one or more of the machine learning models, variables associated with the machine learning models, or both.

[0091] In one example embodiment, initial global upper-level parameters and initial lower-level parameters are transmitted, by the aggregator, to the plurality of client computers to be used in the initial iteration of the federated learning model training.

[0092] In one example embodiment, the assumed ratio of Byzantine clients associated with the lower-level system to non-Byzantine clients associated with the lower-level system is transmitted by the aggregator. It is worth noting that in one or more embodiments, the client(s) does/do not necessarily use the ratio. Rather, this is more in the sense that all clients agree with this assumed ratio to be used by the aggregator for aggregating the hypergradients.

[0093] In one example embodiment, a first upper-level hypergradient is computed, by each of the client computers, using an updated lower-level model, and the first upper-level hypergradient is transmitted to the aggregator.

[0094] In one example embodiment, a global upper-level hypergradient is computed, by the aggregator, using the first upper-level hypergradient, and the global upper-level hypergradient is transmitted to at least one of the plurality of client computers.

[0095] In one example embodiment, a first lower-level hypergradient is computed, by each of the client computers, using an updated lower-level model, and the first lower-level hypergradient is transmitted to the aggregator.

[0096] In one example embodiment, a global lower-level hypergradient is computed, by the aggregator, using the first lower-level hypergradient, and the global lower-level hypergradient is transmitted to at least one of the plurality of client computers.

[0097] In one example embodiment, the generating of the intermediate global lower-level machine learning model and the generating of the global lower-level machine learning model are repeated.

[0098] In one example embodiment, the generating of the intermediate global upper-level machine learning model and the generating of the global upper-level machine learning model are repeated.

[0099] In one example embodiment, the lower-level sets of non-Byzantine clients and a combination of gradients are determined using an aggregation mechanism on first lower-level gradients.

[0100] In one example embodiment, the upper-level sets of non-Byzantine clients and a combination of hypergradients are determined using an aggregation mechanism on client first upper-level hypergradients.

[0101] In one example embodiment, a machine learning inferencing operation is performed using the global upper-level machine learning model.

[0102] In one example embodiment, the performing of the machine learning inferencing operation further comprises detecting a fraudulent financial transaction and mitigating the fraudulent financial transaction. It is worth noting that generally, one or more embodiments can be used where not all the clients are trusted and/or where the communication

network between the clients and the aggregator is unstable, and therefore, the received message sent by the clients can be corrupted.

[0103] In one aspect, a computer program product comprises one or more tangible computer-readable storage media and program instructions stored on at least one of the one or more tangible computer-readable storage media, the program instructions executable by a processor, the program instructions comprising receiving, by an aggregator, updated parameters associated with a lower-level system from a plurality of client computers; generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model by executing federated learning model training and aggregation; generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model by executing the federated learning model training and aggregation; generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model by executing the federated learning model training and aggregation; generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model by executing the federated learning model training and aggregation; and transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

[0104] In one aspect, an apparatus comprises a memory and at least one processor, coupled to the memory, and operative to perform operations comprising receiving, by an aggregator, updated parameters associated with a lower-level system from a plurality of client computers; generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model by executing federated learning model training and aggregation; generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model by executing the federated learning model training and aggregation; generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model by executing the federated learning model training and aggregation; generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model by executing the federated learning model training and aggregation; and trans-

mitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

[0105] In one example embodiment, an inner loop of federated learning model training includes computing an upper-level hypergradient at each client computer and sending the upper-level hypergradients to the aggregator; computing an aggregated upper-level hypergradient at the aggregator and sending the aggregated upper-level hypergradient to the client computers; updating a local upper-level model at each client computer and sending the updated local upper-level model to the aggregator; creating a global upper-level model at the aggregator; and sending the created global upper-level model to the client computers.

[0106] In one example embodiment, an outer loop of federated learning model training includes computing a lower-level hypergradient at each client computer and sending the lower-level hypergradients to the aggregator; computing an aggregated lower-level hypergradient at the aggregator and sending the aggregated lower-level hypergradient to the client computers; updating a local lower-level model at each client computer and sending the updated local lower-level model to the aggregator; creating a global lower-level model at the aggregator; and sending the created global lower-level model to the client computers.

[0107] Refer now to FIG. 13.

[0108] Various aspects of the present disclosure are described by narrative text, flowcharts, block diagrams of computer systems and/or block diagrams of the machine logic included in computer program product (CPP) embodiments. With respect to any flowcharts, depending upon the technology involved, the operations can be performed in a different order than what is shown in a given flowchart. For example, again depending upon the technology involved, two operations shown in successive flowchart blocks may be performed in reverse order, as a single integrated step, concurrently, or in a manner at least partially overlapping in time.

[0109] A computer program product embodiment (“CPP embodiment” or “CPP”) is a term used in the present disclosure to describe any set of one, or more, storage media (also called “mediums”) collectively included in a set of one, or more, storage devices that collectively include machine readable code corresponding to instructions and/or data for performing computer operations specified in a given CPP claim. A “storage device” is any tangible device that can retain and store instructions for use by a computer processor. Without limitation, the computer readable storage medium may be an electronic storage medium, a magnetic storage medium, an optical storage medium, an electromagnetic storage medium, a semiconductor storage medium, a mechanical storage medium, or any suitable combination of the foregoing. Some known types of storage devices that include these mediums include: diskette, hard disk, random access memory (RAM), read-only memory (ROM), erasable programmable read-only memory (EPROM or Flash memory), static random access memory (SRAM), compact disc read-only memory (CD-ROM), digital versatile disk (DVD), memory stick, floppy disk, mechanically encoded device (such as punch cards or pits/lands formed in a major surface of a disc) or any suitable combination of the foregoing. A computer readable storage medium, as that term is used in the present disclosure, is not to be construed as storage in the form of transitory signals per se, such as radio

waves or other freely propagating electromagnetic waves, electromagnetic waves propagating through a waveguide, light pulses passing through a fiber optic cable, electrical signals communicated through a wire, and/or other transmission media. As will be understood by those of skill in the art, data is typically moved at some occasional points in time during normal operations of a storage device, such as during access, de-fragmentation or garbage collection, but this does not render the storage device as transitory because the data is not transitory while it is stored.

[0110] Computing environment 100 contains an example of an environment for the execution of at least some of the computer code involved in performing the inventive methods, such as federated learning aggregator 200 (also generally representative of a federated learning client). In addition to block 200, computing environment 100 includes, for example, computer 101, wide area network (WAN) 102, end user device (EUD) 103, remote server 104, public cloud 105, and private cloud 106. In this embodiment, computer 101 includes processor set 110 (including processing circuitry 120 and cache 121), communication fabric 111, volatile memory 112, persistent storage 113 (including operating system 122 and block 200, as identified above), peripheral device set 114 (including user interface (UI) device set 123, storage 124, and Internet of Things (IoT) sensor set 125), and network module 115. Remote server 104 includes remote database 130. Public cloud 105 includes gateway 140, cloud orchestration module 141, host physical machine set 142, virtual machine set 143, and container set 144.

[0111] COMPUTER 101 may take the form of a desktop computer, laptop computer, tablet computer, smart phone, smart watch or other wearable computer, mainframe computer, quantum computer or any other form of computer or mobile device now known or to be developed in the future that is capable of running a program, accessing a network or querying a database, such as remote database 130. As is well understood in the art of computer technology, and depending upon the technology, performance of a computer-implemented method may be distributed among multiple computers and/or between multiple locations. On the other hand, in this presentation of computing environment 100, detailed discussion is focused on a single computer, specifically computer 101, to keep the presentation as simple as possible. Computer 101 may be located in a cloud, even though it is not shown in a cloud in FIG. 13. On the other hand, computer 101 is not required to be in a cloud except to any extent as may be affirmatively indicated.

[0112] PROCESSOR SET 110 includes one, or more, computer processors of any type now known or to be developed in the future. Processing circuitry 120 may be distributed over multiple packages, for example, multiple, coordinated integrated circuit chips. Processing circuitry 120 may implement multiple processor threads and/or multiple processor cores. Cache 121 is memory that is located in the processor chip package(s) and is typically used for data or code that should be available for rapid access by the threads or cores running on processor set 110. Cache memories are typically organized into multiple levels depending upon relative proximity to the processing circuitry. Alternatively, some, or all, of the cache for the processor set may be located “off chip.” In some computing environments, processor set 110 may be designed for working with qubits and performing quantum computing.

[0113] Computer readable program instructions are typically loaded onto computer 101 to cause a series of operational steps to be performed by processor set 110 of computer 101 and thereby effect a computer-implemented method, such that the instructions thus executed will instantiate the methods specified in flowcharts and/or narrative descriptions of computer-implemented methods included in this document (collectively referred to as “the inventive methods”). These computer readable program instructions are stored in various types of computer readable storage media, such as cache 121 and the other storage media discussed below. The program instructions, and associated data, are accessed by processor set 110 to control and direct performance of the inventive methods. In computing environment 100, at least some of the instructions for performing the inventive methods may be stored in block 200 in persistent storage 113.

[0114] COMMUNICATION FABRIC 111 is the signal conduction path that allows the various components of computer 101 to communicate with each other. Typically, this fabric is made of switches and electrically conductive paths, such as the switches and electrically conductive paths that make up busses, bridges, physical input/output ports and the like. Other types of signal communication paths may be used, such as fiber optic communication paths and/or wireless communication paths.

[0115] VOLATILE MEMORY 112 is any type of volatile memory now known or to be developed in the future. Examples include dynamic type random access memory (RAM) or static type RAM. Typically, volatile memory 112 is characterized by random access, but this is not required unless affirmatively indicated. In computer 101, the volatile memory 112 is located in a single package and is internal to computer 101, but, alternatively or additionally, the volatile memory may be distributed over multiple packages and/or located externally with respect to computer 101.

[0116] PERSISTENT STORAGE 113 is any form of non-volatile storage for computers that is now known or to be developed in the future. The non-volatility of this storage means that the stored data is maintained regardless of whether power is being supplied to computer 101 and/or directly to persistent storage 113. Persistent storage 113 may be a read only memory (ROM), but typically at least a portion of the persistent storage allows writing of data, deletion of data and re-writing of data. Some familiar forms of persistent storage include magnetic disks and solid state storage devices. Operating system 122 may take several forms, such as various known proprietary operating systems or open source Portable Operating System Interface-type operating systems that employ a kernel. The code included in block 200 typically includes at least some of the computer code involved in performing the inventive methods.

[0117] PERIPHERAL DEVICE SET 114 includes the set of peripheral devices of computer 101. Data communication connections between the peripheral devices and the other components of computer 101 may be implemented in various ways, such as Bluetooth connections, Near-Field Communication (NFC) connections, connections made by cables (such as universal serial bus (USB) type cables), insertion-type connections (for example, secure digital (SD) card), connections made through local area communication networks and even connections made through wide area networks such as the internet. In various embodiments, UI device set 123 may include components such as a display

screen, speaker, microphone, wearable devices (such as goggles and smart watches), keyboard, mouse, printer, touchpad, game controllers, and haptic devices. Storage 124 is external storage, such as an external hard drive, or insertable storage, such as an SD card. Storage 124 may be persistent and/or volatile. In some embodiments, storage 124 may take the form of a quantum computing storage device for storing data in the form of qubits. In embodiments where computer 101 is required to have a large amount of storage (for example, where computer 101 locally stores and manages a large database) then this storage may be provided by peripheral storage devices designed for storing very large amounts of data, such as a storage area network (SAN) that is shared by multiple, geographically distributed computers. IoT sensor set 125 is made up of sensors that can be used in Internet of Things applications. For example, one sensor may be a thermometer and another sensor may be a motion detector.

[0118] NETWORK MODULE 115 is the collection of computer software, hardware, and firmware that allows computer 101 to communicate with other computers through WAN 102. Network module 115 may include hardware, such as modems or Wi-Fi signal transceivers, software for packetizing and/or de-packetizing data for communication network transmission, and/or web browser software for communicating data over the internet. In some embodiments, network control functions and network forwarding functions of network module 115 are performed on the same physical hardware device. In other embodiments (for example, embodiments that utilize software-defined networking (SDN)), the control functions and the forwarding functions of network module 115 are performed on physically separate devices, such that the control functions manage several different network hardware devices. Computer readable program instructions for performing the inventive methods can typically be downloaded to computer 101 from an external computer or external storage device through a network adapter card or network interface included in network module 115.

[0119] WAN 102 is any wide area network (for example, the internet) capable of communicating computer data over non-local distances by any technology for communicating computer data, now known or to be developed in the future. In some embodiments, the WAN 102 may be replaced and/or supplemented by local area networks (LANs) designed to communicate data between devices located in a local area, such as a Wi-Fi network. The WAN and/or LANs typically include computer hardware such as copper transmission cables, optical transmission fibers, wireless transmission, routers, firewalls, switches, gateway computers and edge servers.

[0120] END USER DEVICE (EUD) 103 is any computer system that is used and controlled by an end user (for example, a customer of an enterprise that operates computer 101), and may take any of the forms discussed above in connection with computer 101. EUD 103 typically receives helpful and useful data from the operations of computer 101. For example, in a hypothetical case where computer 101 is designed to provide a recommendation to an end user, this recommendation would typically be communicated from network module 115 of computer 101 through WAN 102 to EUD 103. In this way, EUD 103 can display, or otherwise present, the recommendation to an end user. In some

embodiments, EUD **103** may be a client device, such as thin client, heavy client, mainframe computer, desktop computer and so on.

[0121] REMOTE SERVER **104** is any computer system that serves at least some data and/or functionality to computer **101**. Remote server **104** may be controlled and used by the same entity that operates computer **101**. Remote server **104** represents the machine(s) that collect and store helpful and useful data for use by other computers, such as computer **101**. For example, in a hypothetical case where computer **101** is designed and programmed to provide a recommendation based on historical data, then this historical data may be provided to computer **101** from remote database **130** of remote server **104**.

[0122] PUBLIC CLOUD **105** is any computer system available for use by multiple entities that provides on-demand availability of computer system resources and/or other computer capabilities, especially data storage (cloud storage) and computing power, without direct active management by the user. Cloud computing typically leverages sharing of resources to achieve coherence and economies of scale. The direct and active management of the computing resources of public cloud **105** is performed by the computer hardware and/or software of cloud orchestration module **141**. The computing resources provided by public cloud **105** are typically implemented by virtual computing environments that run on various computers making up the computers of host physical machine set **142**, which is the universe of physical computers in and/or available to public cloud **105**. The virtual computing environments (VCEs) typically take the form of virtual machines from virtual machine set **143** and/or containers from container set **144**. It is understood that these VCEs may be stored as images and may be transferred among and between the various physical machine hosts, either as images or after instantiation of the VCE. Cloud orchestration module **141** manages the transfer and storage of images, deploys new instantiations of VCEs and manages active instantiations of VCE deployments. Gateway **140** is the collection of computer software, hardware, and firmware that allows public cloud **105** to communicate through WAN **102**.

[0123] Some further explanation of virtualized computing environments (VCEs) will now be provided. VCEs can be stored as “images.” A new active instance of the VCE can be instantiated from the image. Two familiar types of VCEs are virtual machines and containers. A container is a VCE that uses operating-system-level virtualization. This refers to an operating system feature in which the kernel allows the existence of multiple isolated user-space instances, called containers. These isolated user-space instances typically behave as real computers from the point of view of programs running in them. A computer program running on an ordinary operating system can utilize all resources of that computer, such as connected devices, files and folders, network shares, CPU power, and quantifiable hardware capabilities. However, programs running inside a container can only use the contents of the container and devices assigned to the container, a feature which is known as containerization.

[0124] PRIVATE CLOUD **106** is similar to public cloud **105**, except that the computing resources are only available for use by a single enterprise. While private cloud **106** is depicted as being in communication with WAN **102**, in other embodiments a private cloud may be disconnected from the

internet entirely and only accessible through a local/private network. A hybrid cloud is a composition of multiple clouds of different types (for example, private, community or public cloud types), often respectively implemented by different vendors. Each of the multiple clouds remains a separate and discrete entity, but the larger hybrid cloud architecture is bound together by standardized or proprietary technology that enables orchestration, management, and/or data/application portability between the multiple constituent clouds. In this embodiment, public cloud **105** and private cloud **106** are both part of a larger hybrid cloud.

[0125] The descriptions of the various embodiments of the present invention have been presented for purposes of illustration, but are not intended to be exhaustive or limited to the embodiments disclosed. Many modifications and variations will be apparent to those of ordinary skill in the art without departing from the scope and spirit of the described embodiments. The terminology used herein was chosen to best explain the principles of the embodiments, the practical application or technical improvement over technologies found in the marketplace, or to enable others of ordinary skill in the art to understand the embodiments disclosed herein.

What is claimed is:

1. A computer-implemented method for defending against Byzantine attacks in a federated learning system, the method comprising:

receiving, by an aggregator, from a plurality of client computers, updated parameters associated with a lower-level system;

generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model, by executing federated learning model training;

generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model, by executing the federated learning model training;

generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model, by executing the federated learning model training;

generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model, by executing the federated learning model training; and

transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

2. The computer-implemented method of claim **1**, further comprising:

transmitting, by the aggregator, the intermediate global lower-level machine learning model to at least one of the plurality of client computers;

receiving, by the aggregator, revised parameters associated with the lower-level system from at least one of the plurality of client computers;
 transmitting, by the aggregator, the global lower-level machine learning model to at least one of the plurality of client computers;
 receiving, by the aggregator, updated parameters associated with the upper-level system from at least one of the plurality of client computers;
 transmitting, by the aggregator, the intermediate global upper-level machine learning model to at least one of the plurality of client computers; and
 receiving, by the aggregator, revised parameters associated with the upper-level system from at least one of the plurality of client computers.

3. The computer-implemented method of claim 2, wherein the updated parameters associated with the lower-level system, the revised parameters associated with the lower-level system, the updated parameters associated with the upper-level system and the revised parameters associated with the upper-level system include one or more of the machine learning models, variables associated with the machine learning models, or both.

4. The computer-implemented method of claim 1, further comprising transmitting, by the aggregator, initial global upper-level parameters and initial lower-level parameters to the plurality of client computers to be used in the initial iteration of the federated learning model training.

5. The computer-implemented method of claim 1, further comprising the aggregator transmitting, to the plurality of client computers, the assumed ratio of Byzantine clients associated with the lower-level system to non-Byzantine clients associated with the lower level system.

6. The computer-implemented method of claim 1, further comprising computing, by each of the client computers, a first upper-level hypergradient using an updated lower-level model, and transmitting the first upper-level hypergradient to the aggregator.

7. The computer-implemented method of claim 6, further comprising computing, by the aggregator, a global upper-level hypergradient using the first upper-level hypergradient, and transmitting the global upper-level hypergradient to at least one of the plurality of client computers.

8. The computer-implemented method of claim 1, further comprising computing, by each of the client computers, a first lower-level hypergradient using an updated lower-level model, and transmitting the first lower-level hypergradient to the aggregator.

9. The computer-implemented method of claim 8, further comprising computing, by the aggregator, a global lower-level hypergradient using the first lower-level hypergradient, and transmitting the global lower-level hypergradient to at least one of the plurality of client computers.

10. The computer-implemented method of claim 1, further comprising repeating the generating the intermediate global lower-level machine learning model and the generating the global lower-level machine learning model.

11. The computer-implemented method of claim 1, further comprising repeating the generating the intermediate global upper-level machine learning model and the generating the global upper-level machine learning model.

12. The computer-implemented method of claim 1, further comprising determining the lower-level sets of non-

Byzantine clients and a combination of gradients using an aggregation mechanism on first lower-level gradients.

13. The computer-implemented method of claim 1, further comprising determining the upper-level sets of non-Byzantine clients and a combination of hypergradients using an aggregation mechanism on client first upper-level hypergradients.

14. The computer-implemented method of claim 1, further comprising performing a machine learning inferencing operation using the global upper-level machine learning model.

15. The computer-implemented method of claim 14, wherein the performing of the machine learning inferencing operation further comprises detecting a fraudulent financial transaction and mitigating the fraudulent financial transaction.

16. A computer program product, comprising:

one or more tangible computer-readable storage media and program instructions stored on at least one of the one or more tangible computer-readable storage media, the program instructions executable by a processor, the program instructions comprising:

receiving, by an aggregator, from a plurality of client computers, updated parameters associated with a lower-level system;

generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, an intermediate global lower-level machine learning model, by executing federated learning model training;

generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model, by executing the federated learning model training;

generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model, by executing the federated learning model training;

generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model, by executing the federated learning model training; and

transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

17. A system comprising:

a memory; and

at least one processor, coupled to said memory, and operative to perform operations comprising:

receiving, by an aggregator, from a plurality of client computers, updated parameters associated with a lower-level system;

generating, using data from an identified first set of lower-level non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level

system, an intermediate global lower-level machine learning model, by executing federated learning model training;

generating, using data from an identified second lower-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the lower-level system, a global lower-level machine learning model, by executing the federated learning model training;

generating, using data from an identified first upper-level set of non-Byzantine client computers, of the plurality of client computers, which are characterized as being non-Byzantine for an upper-level system, an intermediate global upper-level machine learning model, by executing the federated learning model training;

generating, using data from a second upper-level set of client computers, of the plurality of client computers, which are characterized as being non-Byzantine for the upper-level system, a global upper-level machine learning model, by executing the federated learning model training; and

transmitting, by the aggregator, the global upper-level machine learning model to at least one of the plurality of client computers.

18. The system of claim **17**, the operations further comprising:

transmitting, by the aggregator, the intermediate global lower-level machine learning model to at least one of the plurality of client computers;

receiving, by the aggregator, revised parameters associated with the lower-level system from at least one of the plurality of client computers;

transmitting, by the aggregator, the global lower-level machine learning model to at least one of the plurality of client computers;

receiving, by the aggregator, updated parameters associated with the upper-level system from at least one of the plurality of client computers;

transmitting, by the aggregator, the intermediate global upper-level machine learning model to at least one of the plurality of client computers; and

receiving, by the aggregator, revised parameters associated with the upper-level system from at least one of the plurality of client computers.

19. The system of claim **17**, the operations further comprising determining the lower-level sets of non-Byzantine clients and a combination of gradients using an aggregation mechanism on first lower-level gradients.

20. The system of claim **17**, the operations further comprising determining the upper-level sets of non-Byzantine clients and a combination of hypergradients using an aggregation mechanism on client first upper-level hypergradients.

* * * * *